

Phase 3: Advanced Feature Engineering Plan

1. Goal & Requirements

The goal of Phase 3 is to take the raw text of the candidate pairs generated in Phase 2 and transform them into highly discriminative numeric feature vectors for LightGBM.

We will extract mathematical, phonetic, semantic, and structural similarities between the `business_name`, `business_address`, and `zip_pin` of each candidate pair.

1.1 Core Requirements

- **Extreme Performance:** Feature extraction must process tens of millions of pairs quickly using C++ backed libraries (rapidfuzz) and list comprehensions (avoiding `pandas.apply`).
- **Missing Value Handling:** Robustly handle missing components using `np.nan` (native to LightGBM) rather than arbitrary constants like `-1`, along with explicit boolean missing flags.
- **Memory Optimization:** Use `float32` for similarities and `int8/bool` for flags, chunking output to Parquet files to prevent memory exhaustion.

2. Advanced Name Similarity Features

We will utilize `rapidfuzz` for high-speed lexical matching, `jellyfish` for phonetic matching, and custom set operations for token overlap.

Feature Name	Algorithm / Library	Purpose
`name_WRatio`	`rapidfuzz.fuzz.WRatio`	Weighted combination of multiple scorers; hig..
`name_token_..`	`rapidfuzz.fuzz.token_set_ratio`	Handles subset relationships (e.g., "Cafe Rom..
`name_token_..`	`rapidfuzz.fuzz.token_sort_rat..`	Handles out-of-order words.
`name_partia..`	`rapidfuzz.fuzz.partial_ratio`	Catches containment ("Acme" in "Acme Global S..
`name_levens..`	`rapidfuzz.distance.Levenshtein`	Character-level substitutions and edits.
`name_jaro_w..`	`rapidfuzz.distance.JaroWinkle..`	Prefix-weighted; excellent for truncated busi..
`name_char3_..`	Custom Set Intersection	High-speed O(len) overlap of 3-grams to avoid..
`name_word_c..`	`abs(len(tokens1) - len(tokens..`	Detects structural mismatch.
`name_length..`	`min(len1, len2) / max(len1, l..`	Detects extreme abbreviations.
`name_sounde..`	`jellyfish.soundex`	Phonetic exact match.
`name_phonet..`	Levenshtein on Phonetic Codes	Captures phonetic similarities even when exac..

(Optional/Future: Semantic Similarity using SentenceTransformer('all-MiniLM-L6-v2') for cross-lingual or synonym matching like "Bakery" vs "Confectionery").

3. Advanced Address Similarity Features

Instead of comparing full raw address strings, we rely on our extracted/normalized components and robust token matching.

Feature Name	Algorithm / Library	Purpose
`addr_levenshtein..`	`rapidfuzz.distance.Leve..`	Fallback full-address similarity.
`addr_token_jacca..`	Custom Set Intersection	Very robust to missing tokens like "floor" or "..
`addr_token_sort_..`	`rapidfuzz.fuzz.token_so..`	Handles scrambled address components.
`addr_zip_match`	Exact String Compare	Strongest single geographic signal (1=match, 0=..
`addr_zip_prefix_..`	First 3 chars compare	Regional match when full ZIP differs.
`addr_state_match`	Exact Compare	Broad geographic constraint.
`addr_tfidf_cosine`	`scipy.sparse` Dot Product	Re-uses Phase 2 TF-IDF vectors for ultra-fast O..

3.1 Missing Data Handling

- Impute missing similarity scores as np.nan.
- Generate explicit boolean flags: addr_missing_flag, addr_zip_missing_flag.

4. Combined & Meta Features

Features that evaluate the joint distribution of names and addresses.

Feature Name	Logic	Purpose
`country_match`	Exact	Safety feature (should always be 1 after b..
`max_name_similarity`	`max(all name features)`	Ensemble signal.
`num_matching_name_t..`	Count of shared tokens	Simple, fast, effective.
`zip_and_name_match`	`zip_match AND name_WRatio..`	High-confidence composite signal.

5. Performance Optimization Strategy

- No pandas.apply: Use pure Python list comprehensions with zip() for string metrics. This achieves maximum speed with rapidfuzz.

```
scores = [fuzz.WRatio(a, b) for a, b in zip(df.name1, df.name2)]
```

- Chunking & Parquet: The resulting DataFrame will be written to disk in chunks of 5 million pairs using pyarrow.parquet. Parquet's columnar compression minimizes disk I/O and speeds up model training.
- Downcasting:
 - Similarity scores -> np.float32
 - Flags/Counts -> np.int8 / bool
- Multiprocessing (If Needed): For the heaviest custom features (like phonetic edit distance), we will utilize multiprocessing.Pool across CPU cores.